

Auto-Scaling Web Infrastructure on AWS

Terraform · Ansible · Python · AWS

Infrastructure as Code	Cloud Architecture	DevOps	High Availability	Auto Scaling
------------------------	--------------------	--------	-------------------	--------------

1. What is this project?

This project provisions, configures, and monitors a fully automated, production-style web infrastructure on Amazon Web Services — built entirely using Infrastructure as Code (IaC). Nothing is created manually. Every AWS resource is defined in code, version-controlled in Git, and reproducible with a single command.

The infrastructure automatically scales up when traffic increases and scales down when it drops — without any human intervention. A Python monitoring script gives a live health report of all running servers at any time.

The core idea: demonstrate the complete DevOps lifecycle — provision infrastructure, deploy an application, and monitor it — using the exact tools that cloud and infrastructure teams use in professional environments.

2. How it works — the full picture

The project is built in four distinct layers, each handled by a different tool:

LAYER 1 — TERRAFORM: PROVISION THE INFRASTRUCTURE

Terraform reads the configuration files (.tf files) and makes the necessary API calls to AWS to create all cloud resources. It manages them as a group — creating, updating, or deleting only what changed since the last run. This is the principle of declarative IaC: you describe the desired end state, and Terraform figures out how to get there.

Resources Terraform creates in this project:

- ▶ **VPC:** VPC (Virtual Private Cloud)
 - ▶ A logically isolated network inside AWS. All project resources live here. Defined with a CIDR block of 10.0.0.0/16, giving 65,536 available IP addresses.
- ▶ **Subnets:** 2 Public Subnets
 - ▶ Placed in two separate Availability Zones (us-east-1a and us-east-1b). Availability zones are physically separate data centres — if one goes down, the other keeps the app running.
- ▶ **IGW:** Internet Gateway
 - ▶ The "door" between the VPC and the public internet. Without it, nothing inside the VPC can be reached from outside.
- ▶ **Route Table:** Route Table
 - ▶ Tells traffic where to go. All internet traffic (0.0.0.0/0) is routed through the Internet Gateway.
- ▶ **Security Groups:** 2 Security Groups
 - ▶ Virtual firewalls. The ALB security group allows public HTTP traffic. The EC2 security group only allows traffic from the ALB — not directly from the internet.
- ▶ **ALB:** Application Load Balancer
 - ▶ Sits in front of all EC2 instances. Receives all incoming requests and distributes them evenly across healthy instances. Performs health checks every 30 seconds — if a server fails, it is automatically removed from rotation.
- ▶ **Target Group:** Target Group

- ▶ The pool of EC2 instances the ALB sends traffic to. The ALB checks the /health endpoint on each instance to verify it is alive.
- ▶ **Launch Template:** Launch Template
 - ▶ A blueprint for every EC2 instance the Auto Scaling Group creates. Defines the instance type (t2.micro), the OS (Amazon Linux 2), security groups, and a bootstrap script.
- ▶ **ASG:** Auto Scaling Group
 - ▶ Automatically adds EC2 instances when CPU exceeds 70%, and removes them when load drops. Configured with min=1, max=3, desired=2. Instances are spread across both availability zones.
- ▶ **Scaling Policy:** Scaling Policy
 - ▶ A Target Tracking policy targeting 70% average CPU utilisation. AWS CloudWatch monitors the metric and triggers scale-out or scale-in automatically.

LAYER 2 — ANSIBLE: CONFIGURE THE SERVERS

Once Terraform creates the EC2 instances, Ansible takes over. It SSH-es into each server and runs a playbook — a YAML file containing ordered tasks. Ansible is idempotent: running the playbook twice produces the same result as running it once. No accidental duplication.

What the Ansible playbook does on each server:

- ▶ Verifies Python 3 and pip are installed (installed by the bootstrap script, Ansible confirms)
- ▶ Installs Flask via pip3
- ▶ Creates the /home/ec2-user/app directory
- ▶ Copies app.py from your local machine to the server using the copy module
- ▶ Creates a systemd service unit file so Flask runs as a background service
- ▶ Enables the service to start automatically on server reboot
- ▶ Starts the service — the Flask app is now live and serving traffic
- ▶ Uses a handler to restart the service only if the app file actually changed — no unnecessary restarts

LAYER 3 — FLASK APP: THE RUNNING APPLICATION

A lightweight Python web application running on each EC2 instance behind the load balancer. Its key feature: it displays its own hostname and private IP address on the page.

When you open the ALB URL and refresh repeatedly, the hostname changes — because the ALB is routing each request to a different EC2 instance. This proves load balancing is working in real time. The app also exposes a /health endpoint that returns HTTP 200 with a JSON payload — used by the ALB for health checks.

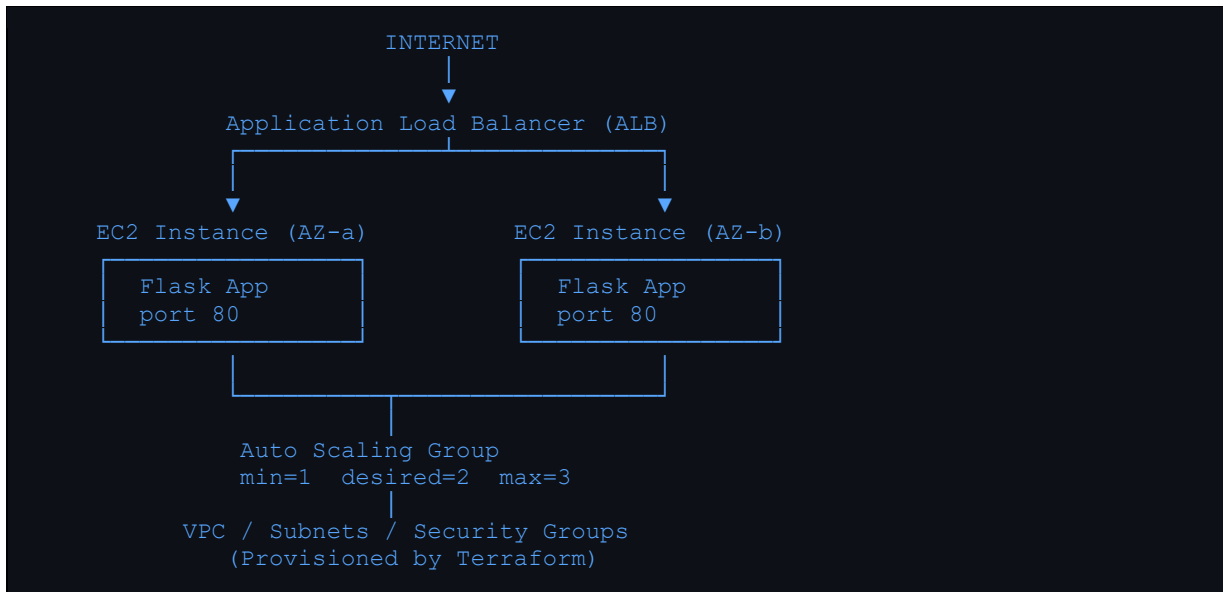
LAYER 4 — PYTHON / BOTO3: MONITOR THE INFRASTRUCTURE

A Python script using boto3 (the AWS SDK) that queries three AWS services simultaneously and prints a live health dashboard:

- ▶ autoscaling client — lists all EC2 instances in the ASG, their lifecycle state (InService, Pending, Terminating), and ASG health status
- ▶ elbv2 client — checks which instances the Load Balancer considers healthy (passing the /health check)
- ▶ ec2 client — fetches each instance's public IP, private IP, instance type, EC2 state, and launch time

The script combines all three into a formatted report showing which servers are healthy, their IPs, and how long they have been running.

3. Architecture at a glance



4. Key concepts this project demonstrates

INFRASTRUCTURE AS CODE (IAC)

Every single AWS resource — the VPC, subnets, load balancer, EC2 instances, security groups, auto scaling group — is defined in Terraform .tf files. No clicking in the console. The entire infrastructure can be destroyed and recreated identically with two commands: terraform destroy and terraform apply. This is the core value of IaC: reproducibility, version control, and auditability.

IDEMPOTENCY

Both Terraform and Ansible are idempotent. Running terraform apply twice does not create duplicate resources — Terraform compares the desired state (your .tf files) against the current state (state file) and only makes the minimum necessary changes. Running the Ansible playbook twice does not reinstall packages that are already installed or restart services that have not changed.

HIGH AVAILABILITY

Instances are spread across two Availability Zones. The ALB is also deployed across both subnets. If an entire AWS data centre (AZ) goes down, the application continues serving traffic from the remaining AZ. This is a fundamental cloud architecture pattern that every cloud engineer must understand.

AUTO SCALING

The Auto Scaling Group monitors average CPU utilisation via CloudWatch. When CPU exceeds 70%, AWS automatically launches new EC2 instances from the Launch Template and registers them with the ALB. When CPU drops, instances are terminated. This happens without any human intervention — the infrastructure adapts to demand automatically.

HEALTH CHECKS AND SELF-HEALING

The ALB calls the /health endpoint on every instance every 30 seconds. If two consecutive checks fail, the instance is marked unhealthy and removed from the load balancer rotation. The ASG detects the unhealthy instance and launches a replacement. The system heals itself without manual intervention.

SEPARATION OF CONCERNS

Each tool has a clearly defined role. Terraform provisions infrastructure. Ansible configures servers. Python monitors the running system. This separation is a professional engineering pattern — it makes each layer independently testable, maintainable, and replaceable.

5. Tools used and why each one

Tool	Category	Why it was chosen
Terraform	IaC / Provisioning	Industry-standard IaC tool. Declarative syntax — you describe what you want, not how to build it. State management tracks real infrastructure. Required skill for this exact role.
Ansible	Config Management	Agentless — no software needed on servers. Uses SSH, which EC2 already supports. YAML playbooks are readable and easy to explain in interviews.
AWS EC2	Compute	Virtual machines that run the Flask app. t2.micro is free-tier eligible, keeping costs at zero during development.
AWS ALB	Load Balancing	Distributes traffic, performs health checks, integrates natively with Auto Scaling Groups.
AWS ASG	Auto Scaling	Manages instance count automatically based on CloudWatch metrics. Core to demonstrating scalable architecture.
Flask	Web App	Minimal Python web framework. Purpose here is not the app itself but proving the infrastructure works — showing which server you are on.
boto3	Monitoring	Official AWS Python SDK. Demonstrates Python + cloud integration, a key good-to-have skill in the JD.

6. Resume content — ready to copy

Use these exactly as written. Each bullet is crafted to match keywords from the Accenture job description while accurately describing what the project does.

OPTION A — SINGLE POWER BULLET (USE THIS IF YOU HAVE LIMITED SPACE)

Designed and deployed an auto-scaling web infrastructure on AWS using Terraform for IaC provisioning and Ansible for configuration management — including VPC, ALB, and Auto Scaling Group across multiple availability zones — monitored with a Python (boto3) health-check script. GitHub: github.com/[your-username]/aws-iac-project

OPTION B — EXPANDED BULLETS (USE THIS IF YOU HAVE A PROJECTS SECTION)

Provisioned complete AWS infrastructure (VPC, subnets, ALB, EC2, ASG, Security Groups) using Terraform, enabling full infrastructure reproducibility from version-controlled .tf files
Automated server configuration and app deployment across multiple EC2 instances using Ansible playbooks, demonstrating idempotent configuration management
Implemented Application Load Balancer with auto-scaling policy targeting 70% CPU utilisation, supporting automatic scale-out/scale-in across two AWS Availability Zones
Built a Python monitoring script using boto3 to query ASG instance lifecycle, ALB target health, and EC2 metadata — providing a live infrastructure health report

Achieved zero-downtime architecture through multi-AZ deployment and ALB health checks that automatically remove unhealthy instances from traffic rotation

TECHNICAL SKILLS LINE — ADD TO YOUR SKILLS SECTION

Cloud & DevOps: Terraform (IaC), Ansible, AWS (EC2, VPC, ALB, ASG, IAM, CloudWatch), Python, boto3, Flask, Git, systemd, Linux (Amazon Linux 2)

7. How to talk about this project in an interview

This is what you say when the interviewer asks "Tell me about a project you have worked on." Practise this until it takes exactly 90 seconds.

"I built an auto-scaling web infrastructure on AWS using Infrastructure as Code.

I used Terraform to provision everything from scratch — a custom VPC, subnets across two availability zones, an Application Load Balancer, and an Auto Scaling Group of EC2 instances. The ASG scales between 1 and 3 instances automatically based on CPU — when load hits 70%, it adds capacity without any manual action.

I used Ansible to configure each server — it SSH-ed into the EC2 instances and deployed a Flask web app as a systemd service. The app shows which server you are being served by, so refreshing the browser proves the load balancer is distributing traffic across different instances.

I also wrote a Python monitoring script using boto3 that queries the Auto Scaling Group and Load Balancer simultaneously and prints a live health report of every running instance.

The entire infrastructure — Terraform files, Ansible playbook, Python script, and app code — is version-controlled on GitHub with a full README."

IF THEY ASK "WHAT WOULD YOU ADD TO MAKE THIS PRODUCTION-READY?"

- ▶ HTTPS with an AWS ACM certificate on the ALB — HTTP is never acceptable in production
- ▶ Remote Terraform state in S3 with DynamoDB locking — prevents corruption in team environments
- ▶ Move EC2 instances to private subnets with a NAT Gateway — servers should not be directly internet-accessible
- ▶ CI/CD pipeline via GitHub Actions — auto-run terraform plan on PRs, auto-apply on merge to main
- ▶ AWS Secrets Manager for credentials — no hardcoded values anywhere in config files
- ▶ CloudWatch alarms with SNS notifications — alert on high CPU, unhealthy hosts, or elevated error rates

8. GitHub profile description

Pin this repo on your GitHub profile. Use this as the repo description (the one-line description under the repo name):

Auto-scaling AWS infrastructure provisioned with Terraform, configured with Ansible, and monitored with Python (boto3). VPC · ALB · ASG · EC2 · IaC

And add these topics/tags to the repo on GitHub (under the repo name → gear icon → Topics):

terraform ansible aws infrastructure-as-code devops python boto3 ec2 auto-scaling cloud